



PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning

Presented by Anton Adelöw

August 18, 2026 — Computer Vision Reading Group, KTH Royal Institute of Technology

The Paper

- **PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning**

Arun Mallya and Svetlana Lazebnik, CVPR 2018. Code:

<https://github.com/arunmallya/packnet>

- **One-sentence summary:** prune a trained network to free up redundant weights, then reuse those freed weights to learn a new task, while keeping every previously learned weight frozen so old tasks never degrade.

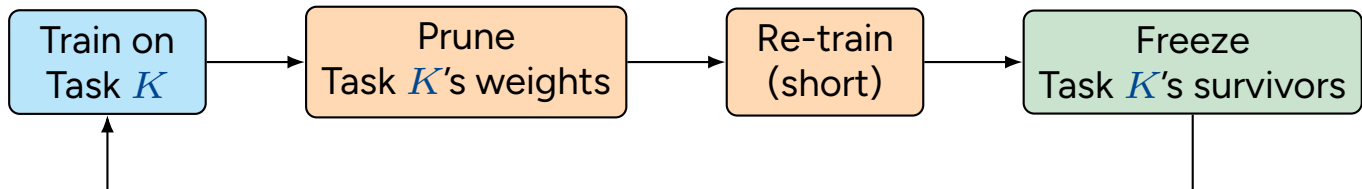
The Continual Learning Problem

- **Goal:** a single network that learns new tasks over time **without** forgetting old ones, but naive fine-tuning causes **catastrophic forgetting**: weights important for old tasks get overwritten.
- Retraining jointly on old and new data isn't always an option: data may be gone (privacy, storage), or the model is already deployed.
- **Setting for this talk:** **task-incremental** learning: the task ID is known at both train and test time. PackNet leans on this directly.

Why Redundancy Helps: The Lottery Ticket Hypothesis

- Contemporaries LwF/EWC fight forgetting with **proxy losses**. PackNet asks a more structural question: **does the network even need all these weights for the current task?**
- **Lottery Ticket Hypothesis** (Frankle & Carbin, ICLR 2019, shortly after PackNet): a dense net contains a sparse **“winning ticket”** subnetwork that matches or exceeds full accuracy when trained alone, found typically by magnitude pruning.
- **Through that lens:** LTH is why PackNet can prune so aggressively without hurting accuracy, dense networks are simply more redundant than they look.

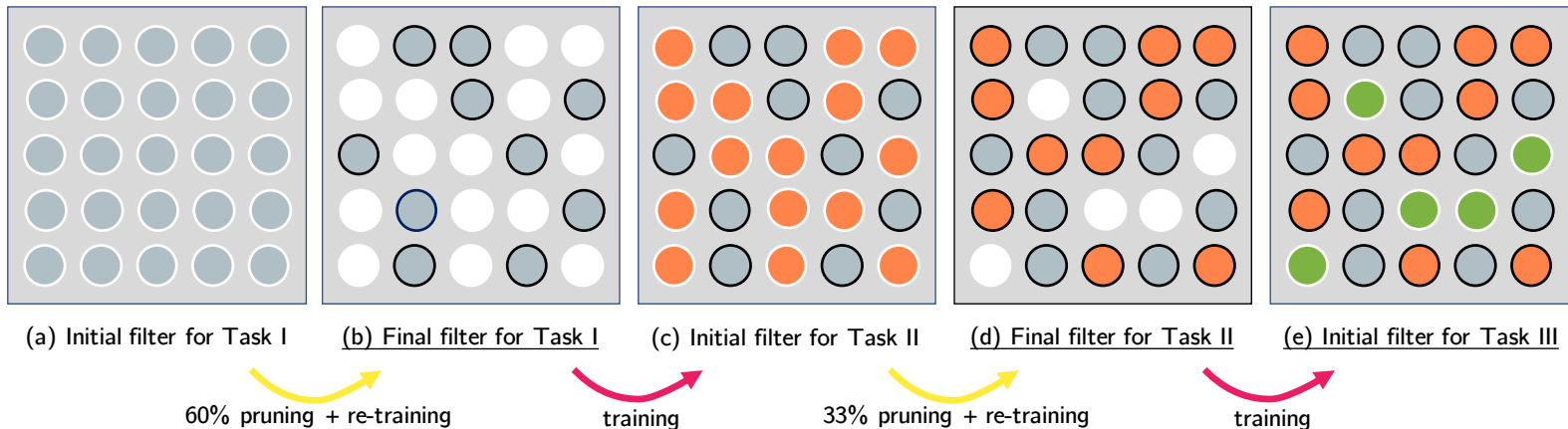
Method Overview



repeat for Task $K + 1$, using freed weights

- Each task only ever modifies weights that **no earlier task** has claimed, giving one physical network with a growing set of per-task binary **masks** that select which weights are “on” for each task.

What Pruning Looks Like, at the Weight Level



- Each round: prune the **current** task's weights, re-train, then **freeze** the survivors forever (gray → orange → green).

The Masking Scheme

- Pruning is **magnitude-based**: within a task's newly trained weights, the smallest 50% or 75% (by absolute value) are zeroed out and handed to the next task.
- Applying a task's mask exactly **reconstructs the network as it looked** right when that task finished training, and only needs to happen **once** per task switch, not per inference call.
- All that needs to be stored is one integer per weight, recording which task first claimed it: $\log_2 N$ bits/weight, not 1 bit/task. For a 537 MB VGG-16, adding 3 tasks costs only ~ 34 MB total.

Adding a Task in Practice

- Every experiment starts from a network **already trained on ImageNet**; that shared backbone is never re-initialized or replaced afterward.
- Adding a new dataset as a task: attach a new output layer (classification head), then run the train, prune, freeze cycle from the Method section.
- This isolates exactly what we want to measure: how well the new dataset is learned, and whether (and by how much) accuracy on the prior task changes, i.e. forgetting.

Experimental Setup

Dataset	# Train	# Eval	# Classes
ImageNet	1,281,144	50,000	1,000
Places365	1,803,460	36,500	365
CUBS Birds	5,994	5,794	200
Stanford Cars	8,144	8,041	196
Flowers	2,040	6,149	102

- Backbone: ImageNet-trained **VGG-16** (also tested: VGG-16+BN, ResNet-50, DenseNet-121).
- Same recipe applies to every backbone; only the network being pruned changes.

Baselines

- **Classifier Only:** freeze the ImageNet backbone, train only a new final layer per task. Cheapest, weakest.
- **Individual Networks:** one fully fine-tuned network per task. Best-case accuracy, worst-case storage ($N \times$ model size).
- **Joint Training:** train on all datasets at once. Only attempted for ImageNet+Places365 (similar dataset sizes); tricky to balance when dataset sizes differ wildly.
- **LwF:** the authors' own re-implementation, same hyperparameters as the original paper.

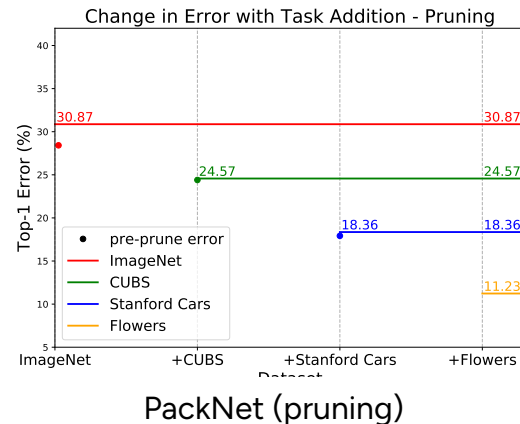
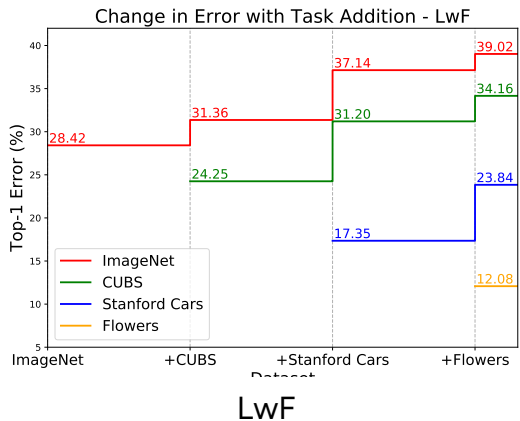
Main Result: Adding Three Fine-Grained Tasks

Dataset	Classifier Only	LwF	Pruning (ours)		Individual Networks
			0.50/0.75/0.75	0.75/0.75/0.75	
ImageNet	28.42	39.23	29.33	30.87	28.42
CUBS	36.76	30.42	25.72	24.95	22.57
Stanford Cars	56.42	22.97	18.08	15.75	13.97
Flowers	20.50	15.21	10.09	9.75	8.65
# Models (Size)	1 (562MB)	1 (562MB)	1 (595MB)	1 (595MB)	4 (2,173MB)

Top-1 error %, averaged over all 6 possible orderings of adding CUBS, Cars, and Flowers.

- Datasets are added sequentially onto the **same** network (pruning header = each task's own pruning fraction, the last task added isn't pruned), beating LwF by a wide margin at nearly the storage cost of a **single** network.

No Forgetting, Visually



- Error on **older** tasks as new tasks are added: LwF climbs every time; PackNet is exactly flat.

Scaling Up: Places365 vs. Joint Training

Dataset	Joint Net.	Pruning		Indiv. Nets
		0.50	0.75	
ImageNet	33.49	29.33	30.87	28.42
Places365	45.98	47.44	46.99	46.35
Size	559MB	576MB	576MB	1,096MB

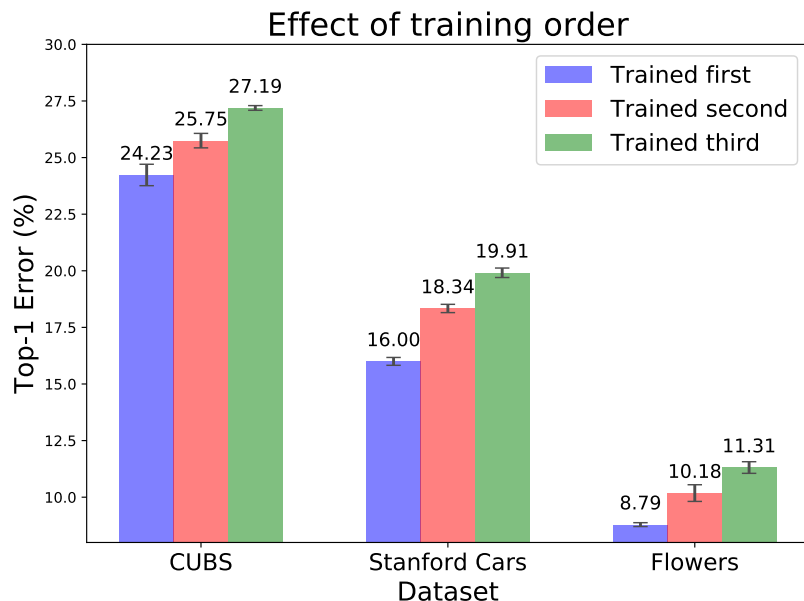
- Places365 (1.8M images) is **larger** than ImageNet, and joint training has no principled way to balance such differently-sized datasets.
- PackNet beats joint training on ImageNet outright (29.33 vs. 33.49), while training for only 10 epochs vs. 60–90.

Generalizing Across Architectures

Architecture	Classifier Only	Pruning (ours)	Individual
	ImageNet top-1 / fine-grained avg. top-1		
VGG-16 + BN	26.63 / 38.09	27.18 / 14.64	26.63 / 13.05
ResNet-50	23.84 / 37.17	24.29 / 13.99	23.84 / 13.68
DenseNet-121	25.56 / 31.22	25.60 / 15.03	25.56 / 13.63

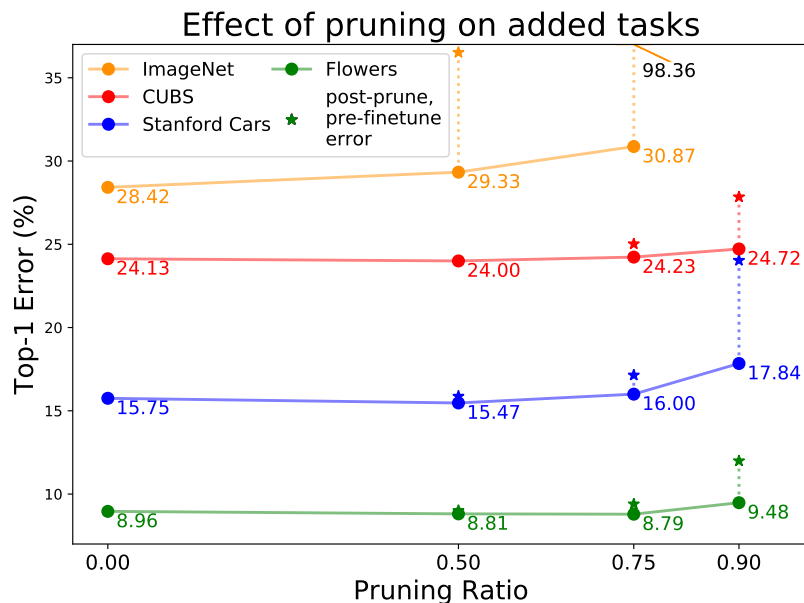
- Works even on much more parameter-efficient, modern architectures with BN, residual, and dense connections, not just the comparatively redundant VGG-16.
- Consistently within 1–2% of the individual-network best case, across architecture families.

Does Task Order Matter?



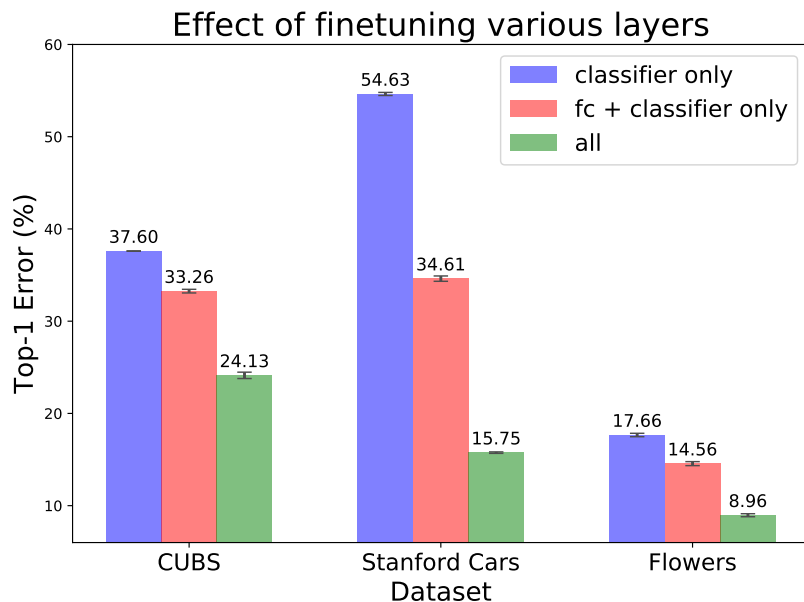
- Free capacity shrinks with every added task: $\sim 134\text{M} \rightarrow 67\text{M} \rightarrow 50.25\text{M} \rightarrow \sim 37\text{M}$.
- So accuracy drops the later a task arrives, e.g. Stanford Cars: $16.00\% \rightarrow 19.91\%$.
- **Takeaway:** add the hardest task **first**, while capacity is plentiful.

Pruning as an Implicit Regularizer



- Pruning **without** re-training always hurts: the ★ markers spike.
- After re-training, error often drops **below** the unpruned baseline: sparsity acts like a regularizer.
- Even a **90% one-shot** prune, after re-training, costs very little (Cars: 15.75%→17.84%).

Which Layers Actually Need to Be Free?



- Start from a 50%-pruned ImageNet network; fine-tune only specific layer groups.
- FC layers alone already beat Classifier-Only, but **convolutional** layers give the biggest gain.
- Runs against the usual “just retrain the classifier head” intuition.

Why Weight-Level, Not Filter-Level, Pruning?

Dataset	Classifier	Filter	Weight
ImageNet	28.42	30.70	29.33
CUBS	36.76	35.73	24.23
Stanford Cars	56.42	34.78	13.97
Flowers	20.50	13.31	8.79

- Filter pruning **would** enable simultaneous multi-task inference.
- But only $\sim 30\%$ of filters could be safely removed, mostly from FC layers, not convolutional ones.
- So filter pruning tops out at **one** additional task in total, versus three for weight pruning, and its accuracy on that one task is worse too.

Strengths and Limitations

Strengths

- Zero forgetting **by construction**
- Near-individual-network accuracy
- Tiny per-task storage overhead
- Inference cost does not grow with more tasks

Limitations

- Needs the task ID at inference time
- No simultaneous multi-task inference
- Masks are heuristic, not learned end-to-end

Summary

- PackNet reframes continual learning as a **capacity allocation** problem: prune a task's redundant weights, freeze the survivors, and hand the freed capacity to the next task, with no proxy losses and no growing architecture.
- The result: near-individual-network accuracy on up to four tasks, on one network, with **zero** forgetting by construction and only a few percent storage overhead.

